



2026-08-30 14:00, ROOM 603

FOSS4G HIROSHIMA 2026

Hands-on CDSE

From the Copernicus Browser
to evalscripts and APIs

Klemen Lovenjak, Sinergise

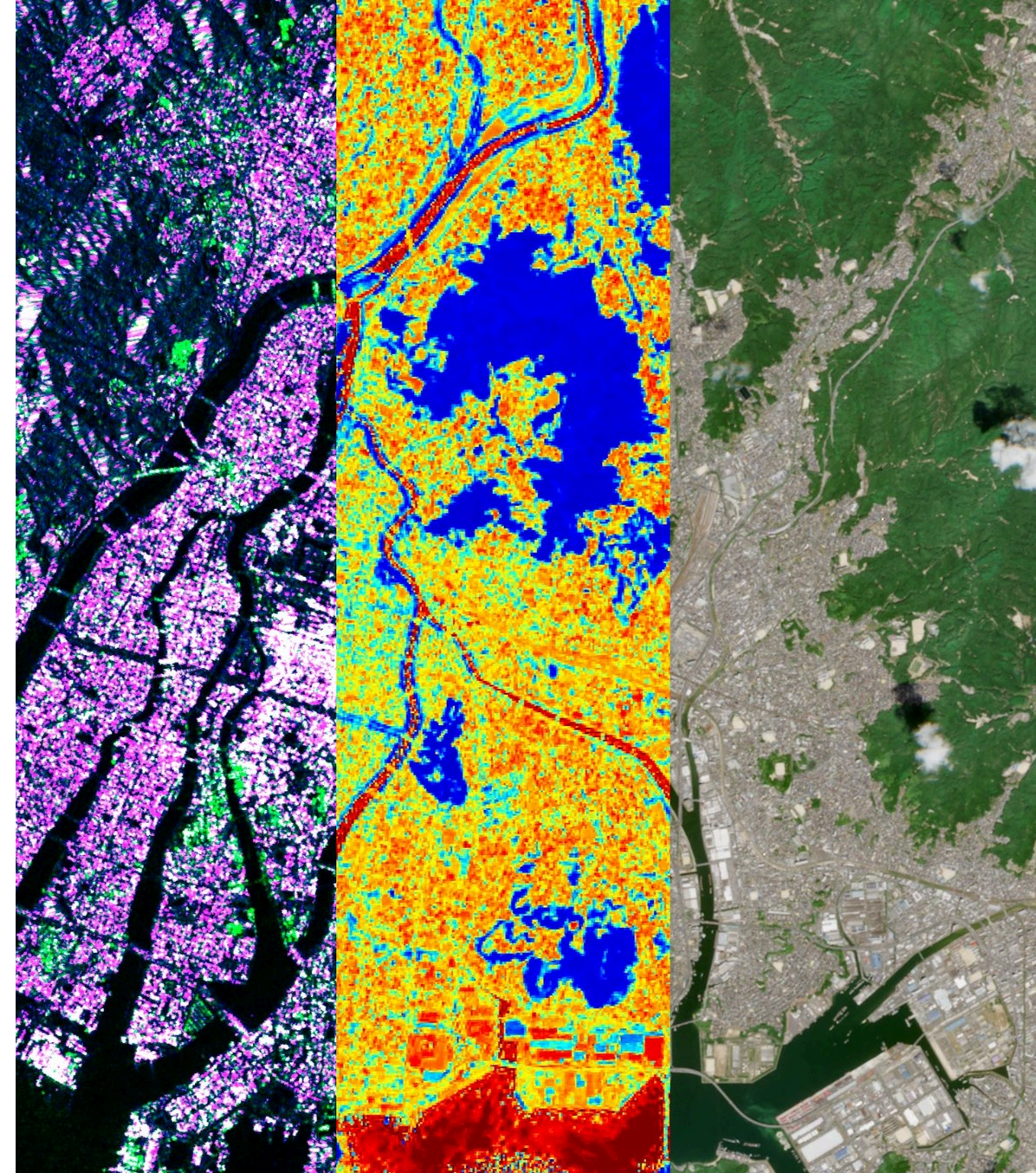


Sentinel-2 over the Adriatic off Emilia-Romagna, Italy. Sediment and algal swirls carried off the Po. Contains modified Copernicus Sentinel data.

CDSE

Europe's Earth observation archive, as a platform you log into.

- Every Sentinel, the whole archive.
Contributing Missions and derived products too.
- A European programme with a **global archive**,
so search wherever you want.
- One archive, two front doors:
the **Browser** and the **APIs**. Today you use both.
- **A free account on the same endpoints production runs on.**
What a bigger tier buys is more of it, not something different.
 - 10 000 requests and 10 000 processing units a month, 300 a minute.
A unit is a **512 × 512 output at three bands**,
which is about 26 km² at Sentinel-2 resolution.



The Ōta delta, Hiroshima, three ways: Sentinel-1 SAR-URBAN, the moisture index, and true colour.
Contains modified Copernicus Sentinel data 2026.

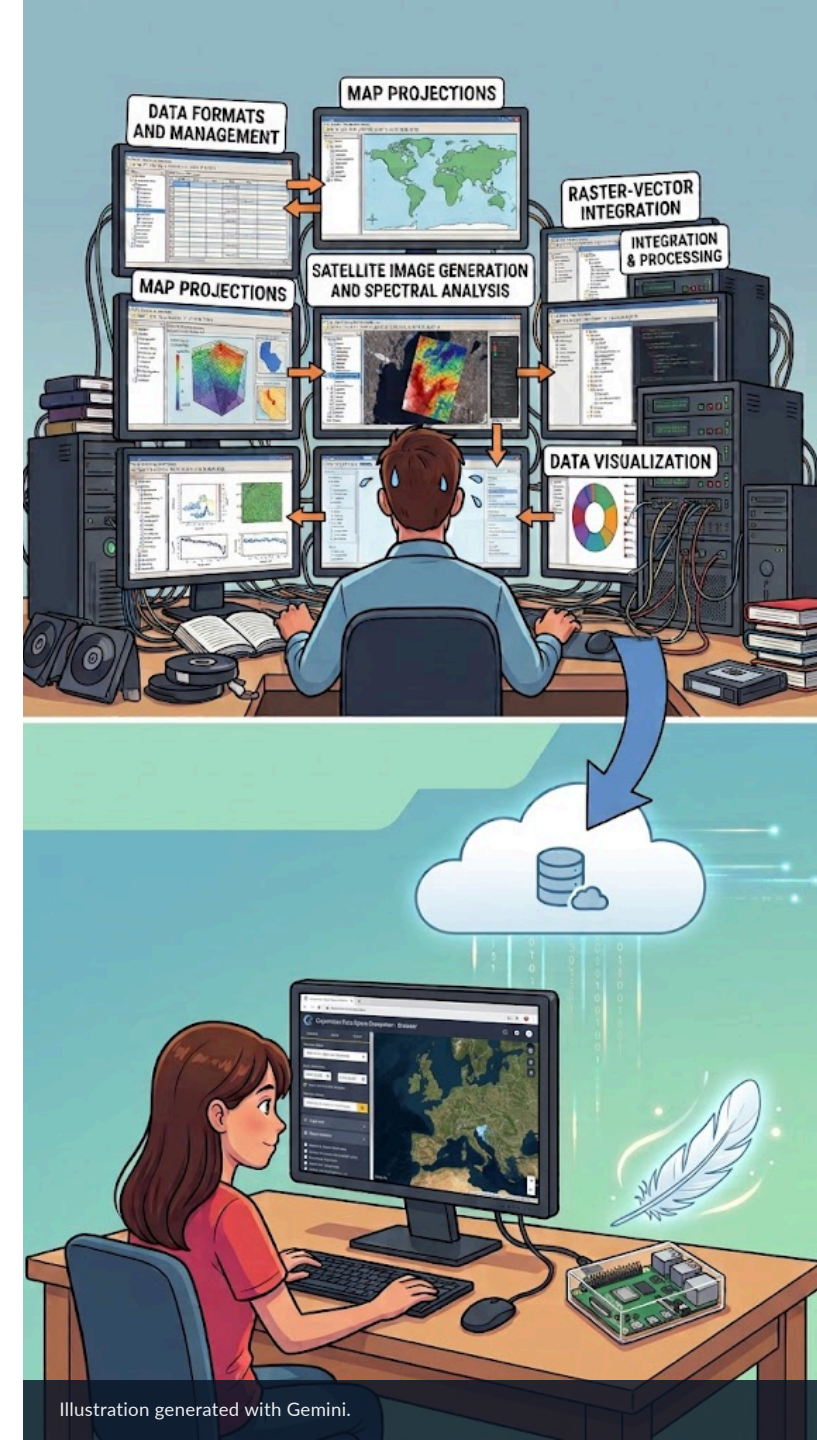
EO coding is difficult

- **A wide range of skills required**

- Data formats and management
- Map projections
- Satellite image generation and spectral analysis
- Raster-vector integration
- Data visualization

- **CDSE turns this into a series of incremental steps**

- Data management is done by the backend
- Visualization is done by the Browser
- You can focus on the core part



Europe's entire EO archive. Free.

- Every Sentinel, the whole archive
- Contributing Missions and derived products
- Land cover, soil moisture, snow, temperature, water quality, burned area, evapotranspiration...
- **Over 90 PB today, more than 100 PB by 2028**
- **Operated by a European consortium under ESA, on European soil.**

MISSION	WHAT IT SEES
Sentinel-1	Through cloud, day or night
Sentinel-2	Land surface, 13 optical bands
Sentinel-3 OLCI	Ocean and land colour
Sentinel-3 SLSTR	Surface temperature
Sentinel-5P	Atmospheric gases

FIVE STOPS

Browser → Catalogue → Evalscript → AI → Python

Clicking, for the first hour.

The Browser, the catalogue, and what an API actually is.

Writing, after the break.

Evalscripts by hand first, then with the AI.

Running it yourself, at the end.

The same script, from Python, over anywhere on Earth.

Do this now

① Log in at **dataspace.copernicus.eu**

② Then open **link.dataspace.copernicus.eu/pkrg**

③ 👍 when you can see **your name, top right**

STOP 1 OF 5

Browser

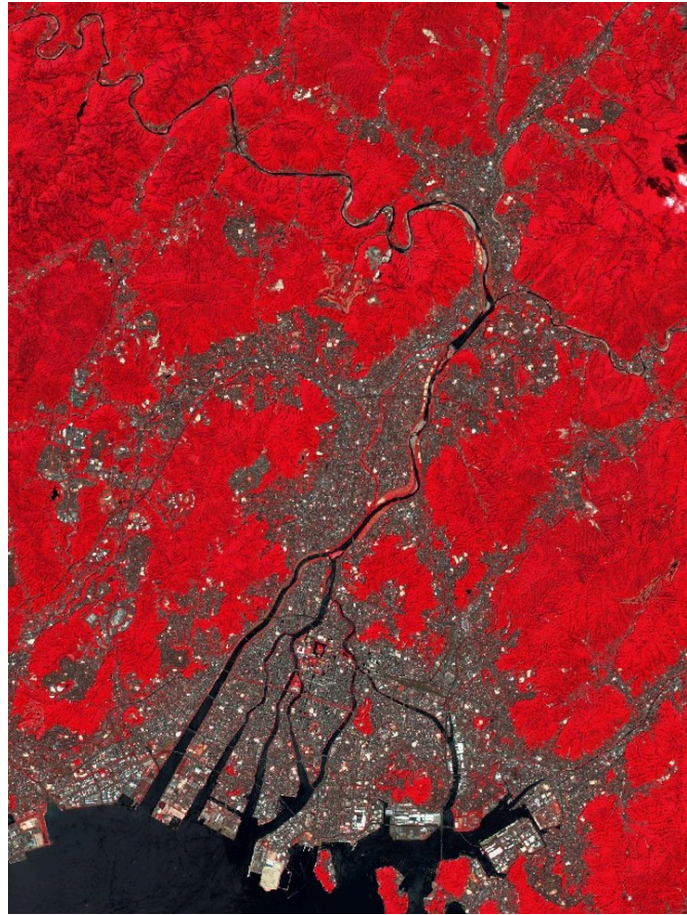
Everything Copernicus has, over anywhere you like, in a browser tab you are already logged into.



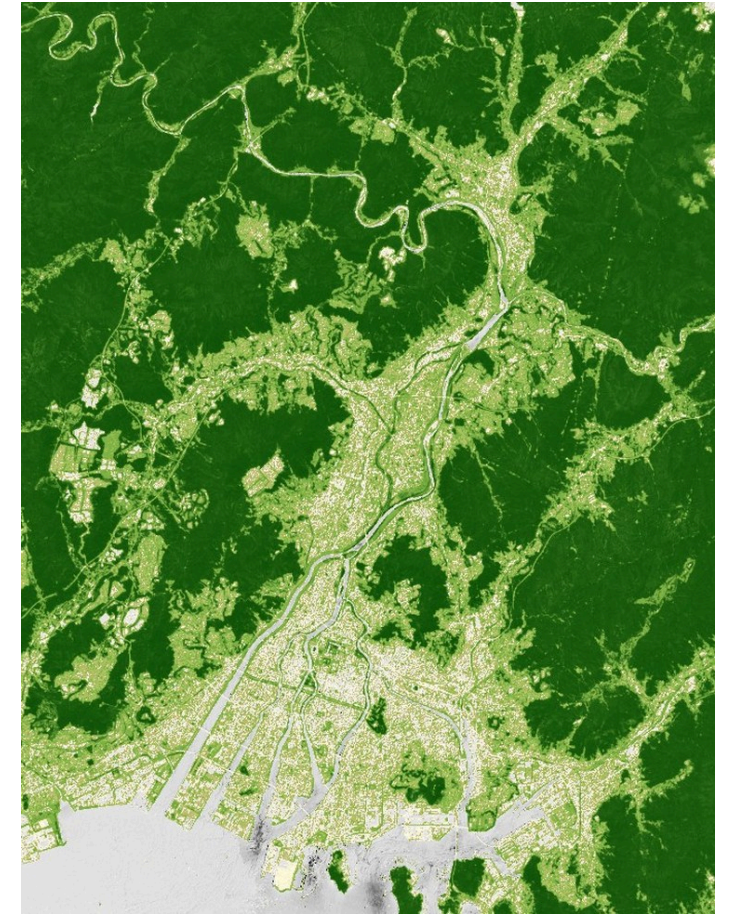
Same place, three lenses



True color
what your eye would see

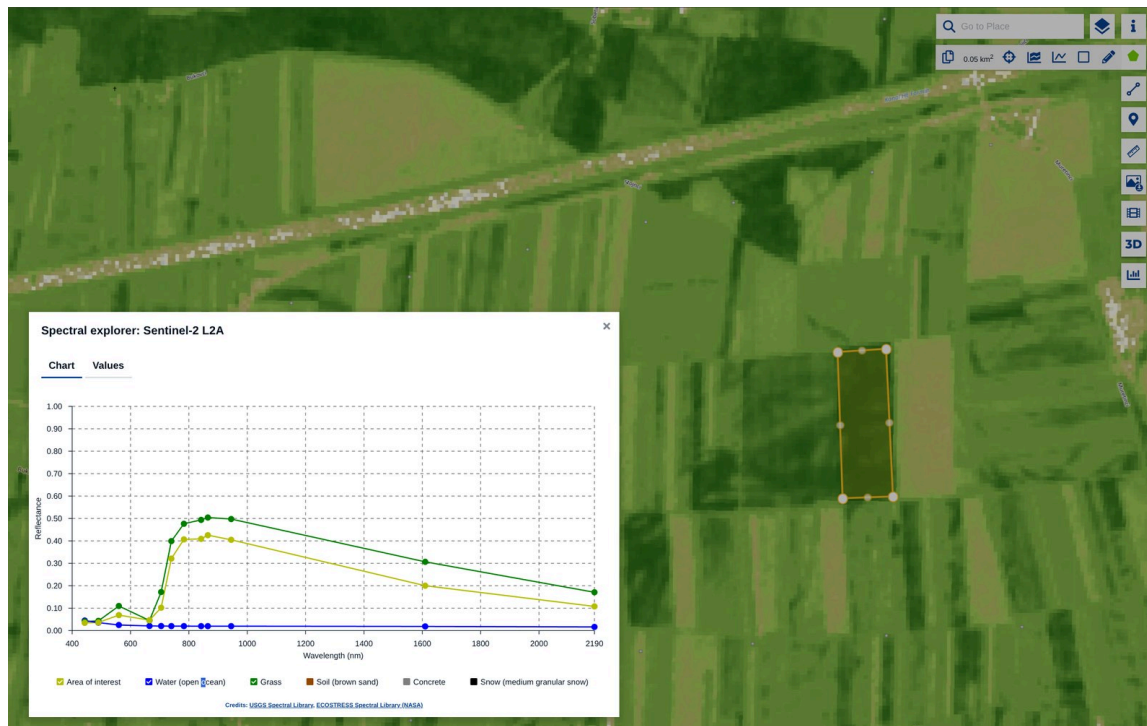


False color
vegetation in red



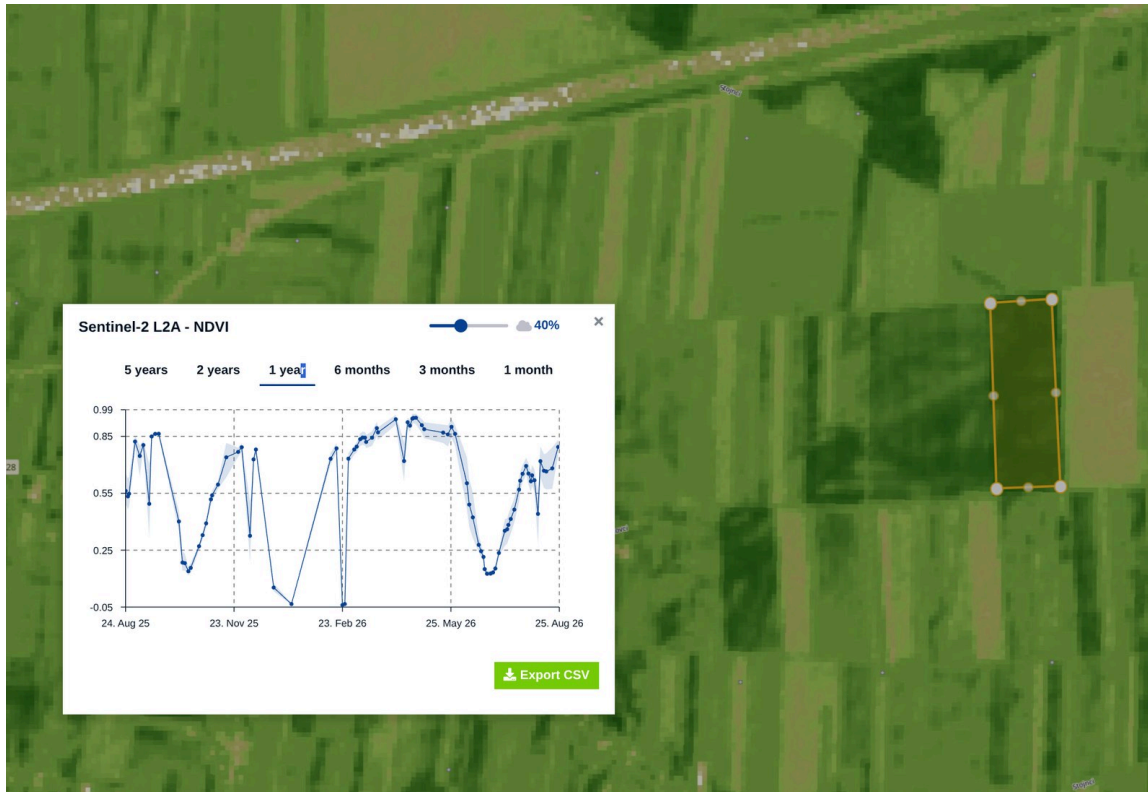
NDVI
every pixel is now a number

Underneath the pictures



- One pixel from the marker, or the mean over a **polygon** you draw
- Vegetation, water and concrete have **different shapes**
- A preset is just a choice of which bands to draw
- **These band names are what you write in an evalscript.**

And a history



- The same marker. **A year of NDVI**, not one date
- The peak is the greenest day, answered **without downloading anything**
- The chart is spiky. Hold that thought.
- **This is the same endpoint you call from Python after the break.**

The whole archive, not a basemap

The screenshot shows the Copernicus Browser interface. At the top, there's a navigation bar with 'EN', 'Klemen Lovenjak', and a back arrow. Below this is a 'VISUALISE' button and a 'SEARCH' button. Underneath are links for 'SH DASHBOARD' and 'WORKSPACE'. The main section is titled 'DATE: SINGLE' and features a date selector set to '2026-08-23' with a '30%' cloud coverage indicator. A green button labeled 'Show latest date' is present. Below this is a link 'Find products for current view'. A slider for 'Max. cloud coverage:' is set to 30%. At the bottom, a calendar for August 2026 is displayed, with the 25th highlighted.

Su	Mo	Tu	We	Th	Fr	Sa
26	27	28	29	30	31	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	1	2	3	4	5

Max. cloud coverage: 30%
Few dates survive it.



Or take a date and process it.
No cloud handling at all.

That 30% is a ~12,000 km² tile average, and shadows are not in it.
Tile-level filter, *Most recent* ordering. **Nothing here knows what a cloud is.**

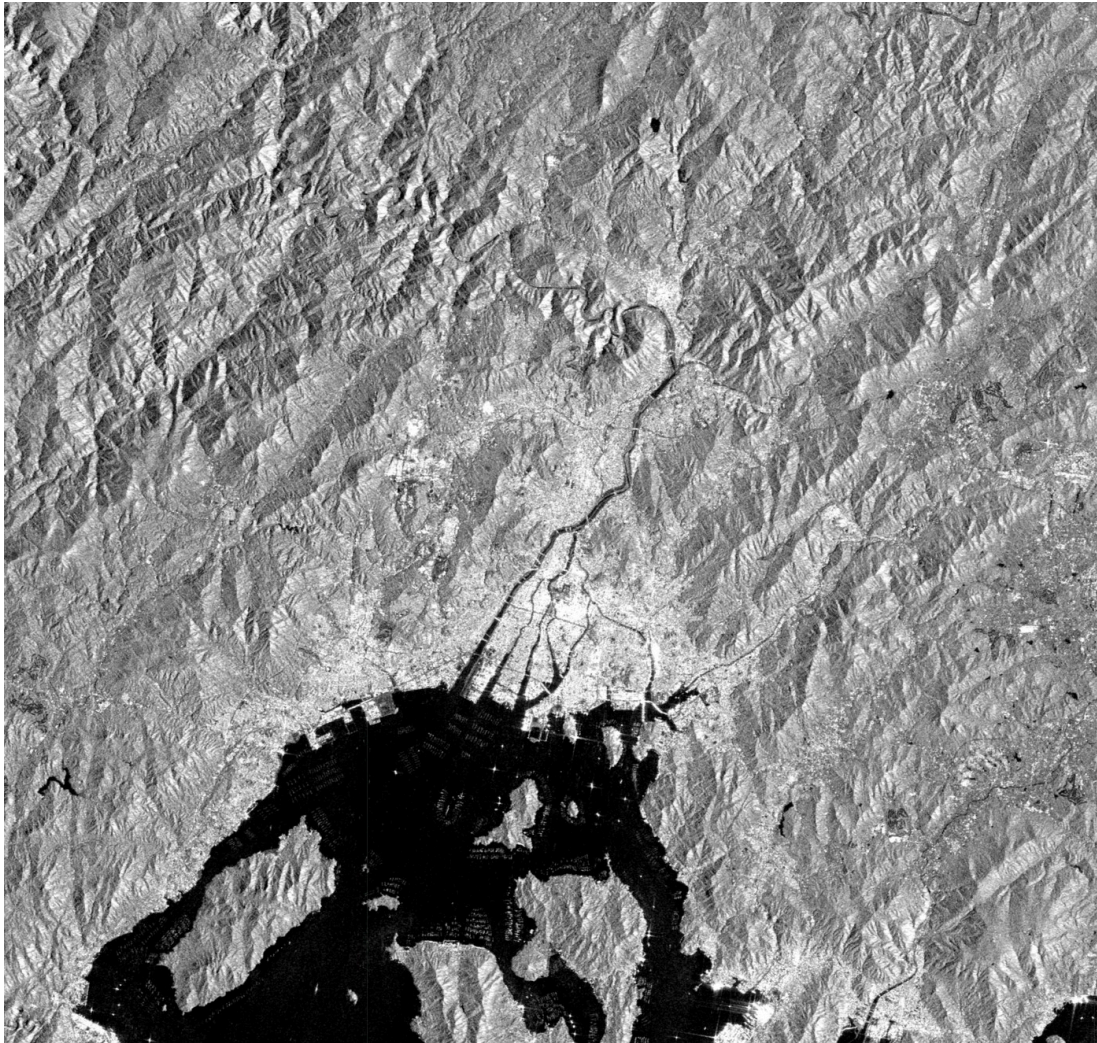
Date: Time Range

Six months of passes, shown as one image.



- **Max. cloud coverage** discards whole tiles, on scene metadata covering $\sim 12,000 \text{ km}^2$ each
- So a tile that is 10% cloudy **passes the filter** with all of that cloud sitting on your area
- **Find products within selected time range** lists the acquisitions themselves, one at a time
- **No individual pixel was checked, so this cannot tell you which are cloud, shadow or ground.**

Same week. Same rain. No cloud problem.



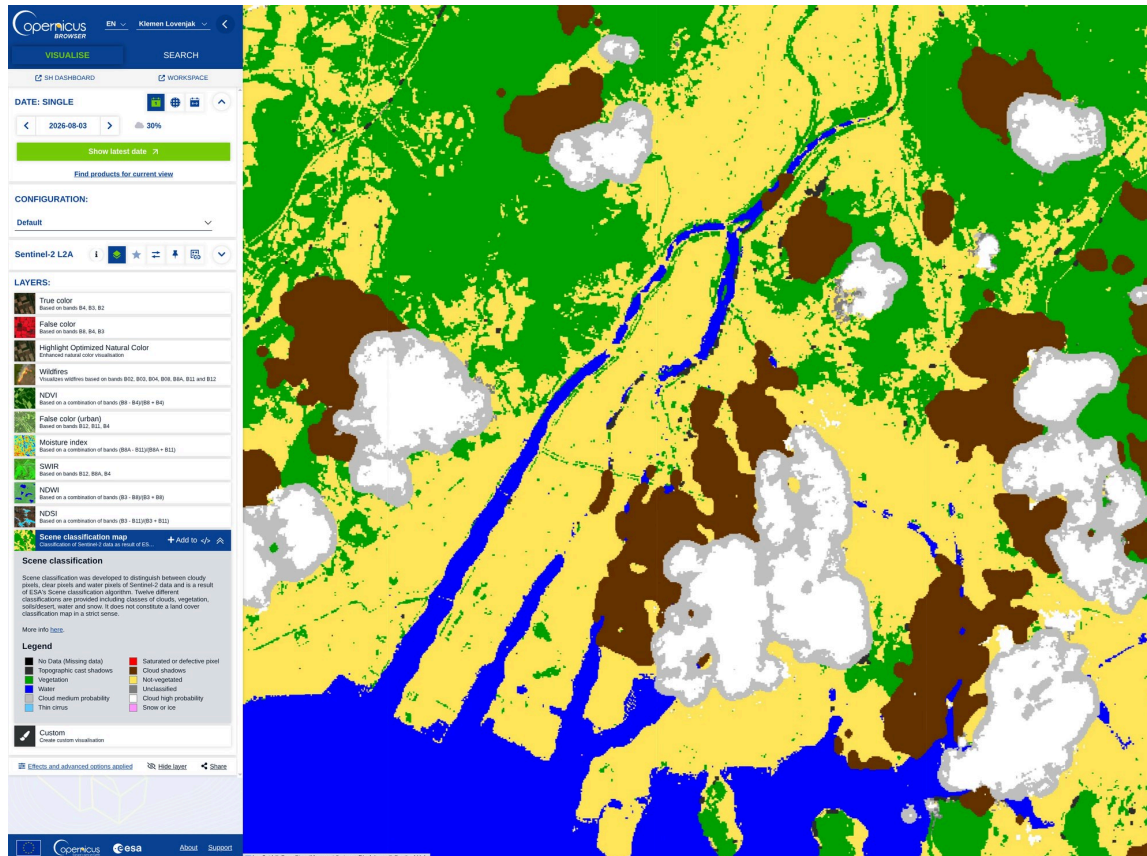
- Radar makes its own light, at a wavelength that goes **straight through water droplets**
- **Bright** is rough or metallic. **Dark** is smooth, which is why the bay is black
- The cost: no colour, no NDVI, speckle, poor geometry in mountains
- **Thermal is blocked by cloud too.** Only the radar walks through
- **Not a free upgrade. A different question.**
The archive is not one satellite.

Your turn

Search for a place of interest, then go and look.

- Switch **layers**. True color, false color, NDVI, moisture
- Drag the **time slider** and watch the season move
- Try the other **Sentinel collections**
- **No target. Get familiar with the Copernicus Browser and the imagery.**
- Drop a **Point of interest**, or draw a **polygon**
- **Statistical Info** for one place over a year
- **Spectral explorer** for the curve under a pixel

Per pixel, not per tile



- Every pixel gets a class. **Twelve** of them.
- Look at the white cloud blobs.
Each has a **brown shadow** beside it.
- Computed by ESA, shipped with the scene
- **This** is what per-pixel means.
The cloud filter you just used was per *tile*.
- **Something still has to decide which of these classes count as unusable.**

STOP 2 OF 5

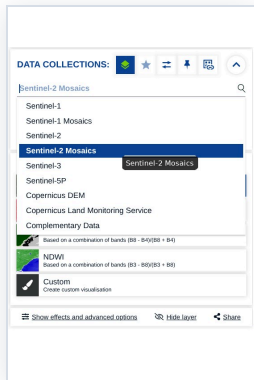
Catalogue

Somebody has probably solved this already,
more than once, and differently each time.

Find out what already exists
before you write anything.

Quarterly cloudless mosaics

Somebody has already composited the season, and they did check every pixel.



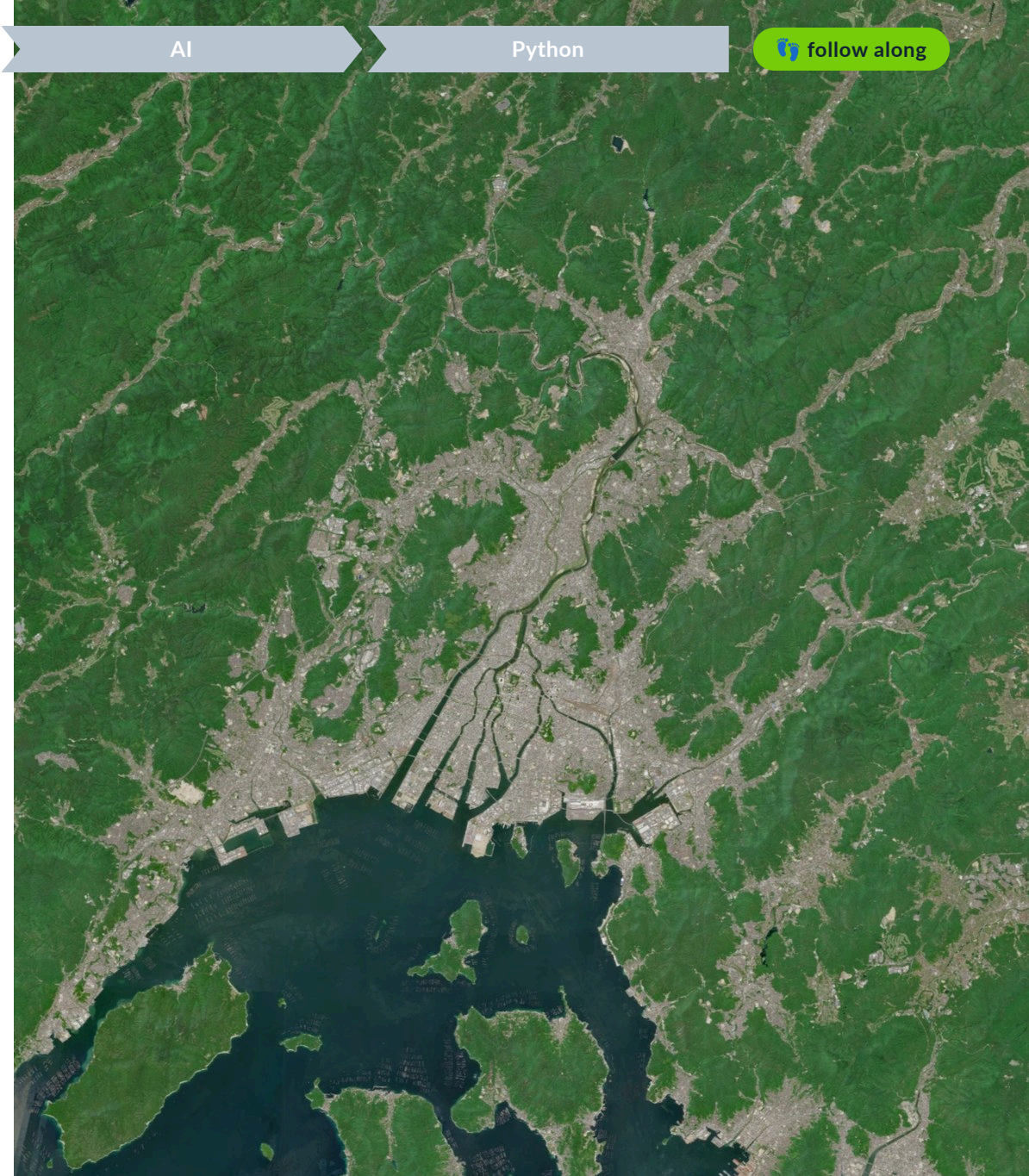
Data collections



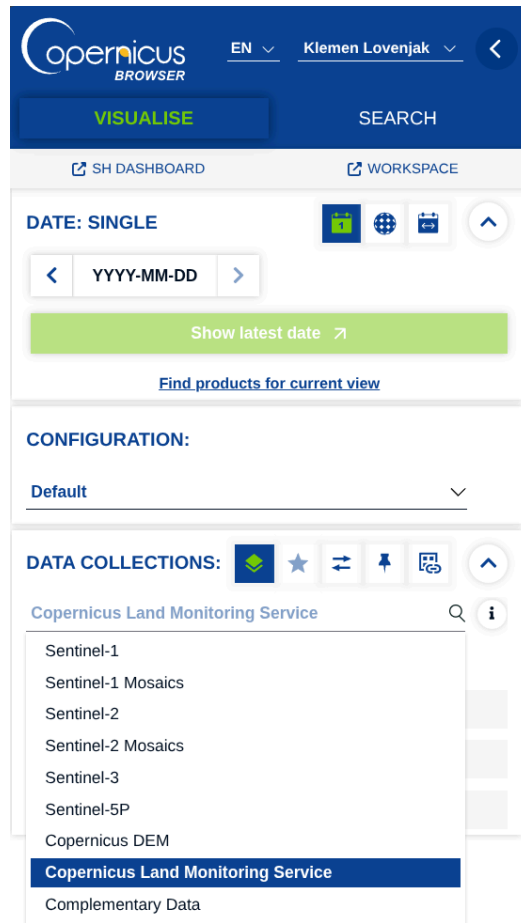
Sentinel-2
Mosaics

- A **separate collection**, not a date setting
- Per pixel over a **three-month stack**
- Rejected on SCL 1, 3, 7, 8, 9, 10, then the **first quartile** of the rest
- There are **three** mosaic collections, and they each pick pixels differently
- **Which is why the shadows went too.**

The algorithm documentation: "No pre-filtering (e.g. based on cloud coverage) was performed to preserve as many non-cloudy pixels as possible."



Nine families in one dropdown



- The **Sentinels** you already know, plus their **Mosaics**
- **Copernicus DEM**, elevation
- **Complementary Data: Landsat 8-9** and **Landsat Mosaics**, which reach back to **1997**, long before Sentinel-2
- **And one that is not a satellite mission at all.**

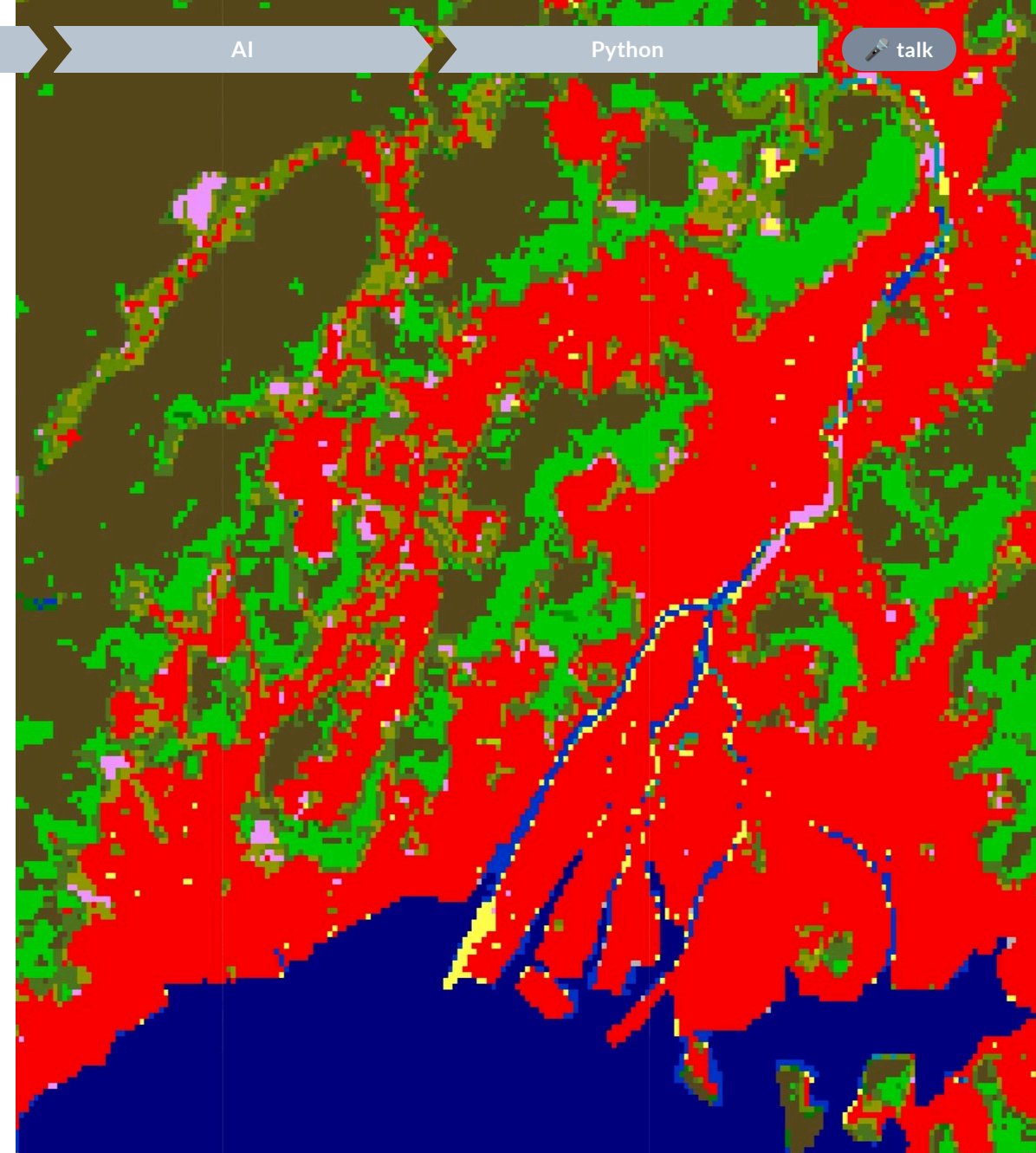
The **Copernicus Land Monitoring Service** ships finished products rather than raw scenes.

CLMS

Copernicus Land Monitoring Service

- Run by the **European Environment Agency** and the **European Commission's Joint Research Centre**, since **2012**
- Land cover, soil moisture, snow, surface temperature, water quality, burned area, phenology, crop type...
- **Not only Sentinel-based**. Also **PROBA-V**, **SPOT**, **Terra** and geostationary sensors, with series back to **1999**
- Which is why cadence is not a revisit. There are **hourly** products, because geostationary is not a passing satellite
- **Because these are not Sentinel scenes, somebody has already decided which observations were valid.**

Everything CLMS makes lives at land.copernicus.eu.
CDSE carries a growing subset of it.



`clms_global_lcc_100m_v3_yearly_geotiff` at Discrete classification, FAO LCCS, 100 m

Faster than you would expect

Global, and current.

WHAT IT IS	COLLECTION ID	CADENCE
Land surface temperature	lst_global_3km_hourly_v3	hourly, 3 km
Vegetation index	ndvi_global_300m_daily_v3	10-daily, 300 m
Burnt area	ba_global_300m_daily_v4	daily, 300 m
Sensible and latent heat flux	hf_global_300m_daily_v1	daily, 300 m
Snow cover extent	sce_global_1km_daily_v1	daily, 1 km

Hourly comes from **imagers on geostationary satellites**, mosaicked together.
The 3 km is **0.025°**, so it is 3 km at the equator and coarser away from it.

Already computed

Finished products. The hard part is knowing they exist.

WHAT IT IS	COLLECTION ID	DETAIL
Actual evapotranspiration	eta_global_300m_10daily_v1	300 m
Lake water quality	lwq-nrt_global_100m_10daily_v2	100 m
Lake surface water temperature	lswt_nrt_global_1km_10daily_v1	1 km
Start and end of growing season	clms_global_lsp_300m_v2_yearly_geotiff	300 m, since 2014
Bare soil days before the crop	cpbsb_10m_yearly_v1	10 m, since 2017
Water bodies	clms_global_wb_100m_v1_monthly_geotiff	100 m, monthly

 These are **Browser ids**. CDSE STAC names the same products differently.

Go and look

Your own place. Whatever looks useful.

- Open **Data collections** and scroll the whole list
- Put a **CLMS product** over somewhere you know
- Zoom in until you can see the pixels
- **Get familiar with the products. Think about what you could use them for in the exercises later today.**
- Read an **id** and guess the product from it
- Find two products that should agree, then check
- Follow a **documentation link** for one of them

Every CLMS product has a page: what it measures, its bands, its scope and its cadence.
`documentation.dataspace.copernicus.eu/APIs/SentinelHub/Data/CLMS.html`

`ask.copernicus.eu` will answer questions about the archive.
It is a bot, so check anything you lean on.

Take the list **with you**

- Every global collection, with its **collection id**
- **Band tables**, so you know what a layer holds before you open it
- A **documentation link** for each product

github.com/lovenjak/hands-on-cdse

⚠ These are **Browser ids**. CDSE STAC names the same products differently, so one will not work in the other.

Five versions of the same ten days

The Browser calls it the consolidation period.

LEVEL	WHAT WENT INTO IT
RT0	The first estimate, as soon as the dekad ends
RT1	Recomputed after one more dekad of data
RT2	After two
RT6	After six . The consolidated one

- Same product, same ten days, **recomputed as more observations arrive**
- Each level is a **separate collection with its own id**.
There is often no plain version to fall back on
- **Faster, or better. The catalogue makes you choose.**

The official metadata says it plainly: *"provided in Near Real Time and consolidated in the next six periods."*

STAC

The same catalogue,
as an open standard.

- `stac.dataspace.copernicus.eu/v1/spec 1.1.0`, 200+ collections
- `/search` answers **without a login**.
No token, no OAuth client, no account
- **Every collection you clicked through is queryable from a script.**

The screenshot shows the web interface of the Copernicus Data Space Ecosystem (CDSE) - STAC API. The header includes navigation links for 'Browse', 'Search', and 'API', along with the CDSE logo and 'STAC API' text. On the right, there are links for 'Log in' and 'Language: English'. The main content area is titled 'Copernicus Data Space Ecosystem (CDSE) - STAC API' and includes a 'Description' section explaining the catalog's purpose and a 'Featured' section displaying five data collection cards. Each card features a satellite image, a title, a brief description, and a timestamp. The 'Additional Resources' section on the right provides links to the OpenAPI service description and documentation.

Copernicus Data Space Ecosystem (CDSE) - STAC API

Description
A comprehensive and searchable catalog of Earth observation and scientific datasets that is actively maintained and updated by the Copernicus Data Space Ecosystem. The integrated STAC (SpatioTemporal Asset Catalog) API, which follows the open STAC specification, enables users to access detailed asset-level metadata, including spatial information, temporal coverage, and data specifications. This standardized interface facilitates efficient data discovery and retrieval across the extensive collection of environmental and scientific resources. More information: <https://documentation.dataspace.copernicus.eu/>

Additional Resources

- [OpenAPI service description](#)
- [OpenAPI service documentation](#)

Featured

- Sentinel-2 Level-2A**
The Sentinel-2 Level-2A Collection 1 product provides orthorectified Surface Reflectance (Bottom-Of-Atmosphere: BOA), with sub-...
2015-06-27 10:25:31 UTC until present
- Sentinel-2 Global Mosaics**
The mosaics have four bands of data (Red (B04), Green (B03), Blue (B02) and wide band Near Infrared (B08)). First, cloud...
2020-01-01 0:00:00 UTC until present
- Sentinel-1 Ground Range Detected (GRD)**
Level-1 Ground Range Detected (GRD) products consist of focused SAR data that has been detected, multi-looked and...
2014-10-04 3:12:47 UTC until present
- Sentinel-1 Global Mosaics**
Sentinel-1 monthly mosaics are generated from monthly stacks of Sentinel-1 GRD data by calculating the weighted sum of the...
2020-01-01 0:00:00 UTC until present
- Sentinel-1 Single Look Complex: IW, EW, SM**
This collection offers Sentinel-1 Level-1 Single Look Complex (SLC) products captured in the modes Interferometric Wid...
2014-10-04 3:06:54 UTC until present

⚠ Discovery is open. **Pixels are not.** Asset hrefs are `s3://` or OIDC-gated, so this is a catalogue, not a download pipe.

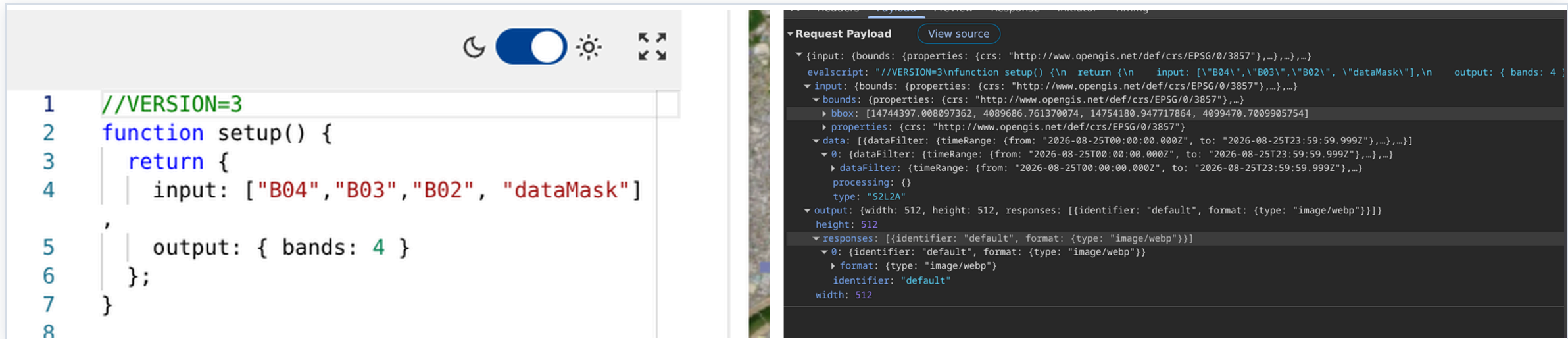
STOP 3 OF 5

Evalscript

Send the code to the **data**, not the data to the code.

You have been doing it since two o'clock.
You just did not write any of it yet.

You have been doing this since the tour started



```

1  //VERSION=3
2  function setup() {
3    return {
4      input: ["B04", "B03", "B02", "dataMask"]
5    },
6    output: { bands: 4 }
7  };
8

```

```

Request Payload
View source
{
  input: {
    bounds: {
      properties: {
        crs: "http://www.opengis.net/def/crs/EPSG/0/3857", ...
      }
    },
    evalscript: "//VERSION=3\nfunction setup() {\n  return {\n    input: [\n      \"B04\", \"B03\", \"B02\", \"dataMask\"\n    ],\n    output: { bands: 4\n  }\n}\n", ...
  },
  input: {
    bounds: {
      properties: {
        crs: "http://www.opengis.net/def/crs/EPSG/0/3857", ...
      }
    },
    bbox: [14744397.008097362, 4089686.761370074, 14754180.947717864, 4099470.7009905754],
    properties: {
      crs: "http://www.opengis.net/def/crs/EPSG/0/3857"
    },
    data: [
      {
        dataFilter: {
          timeRange: {
            from: "2026-08-25T00:00:00.000Z", to: "2026-08-25T23:59:59.999Z", ...
          }
        },
        processing: {
          type: "S2L2A"
        }
      }
    ],
    output: {
      width: 512, height: 512, responses: [
        {
          identifier: "default", format: {
            type: "image/webp"
          }
        }
      ]
    },
    responses: [
      {
        identifier: "default", format: {
          type: "image/webp"
        }
      }
    ],
    format: {
      type: "image/webp"
    },
    identifier: "default",
    width: 512
  }
}

```

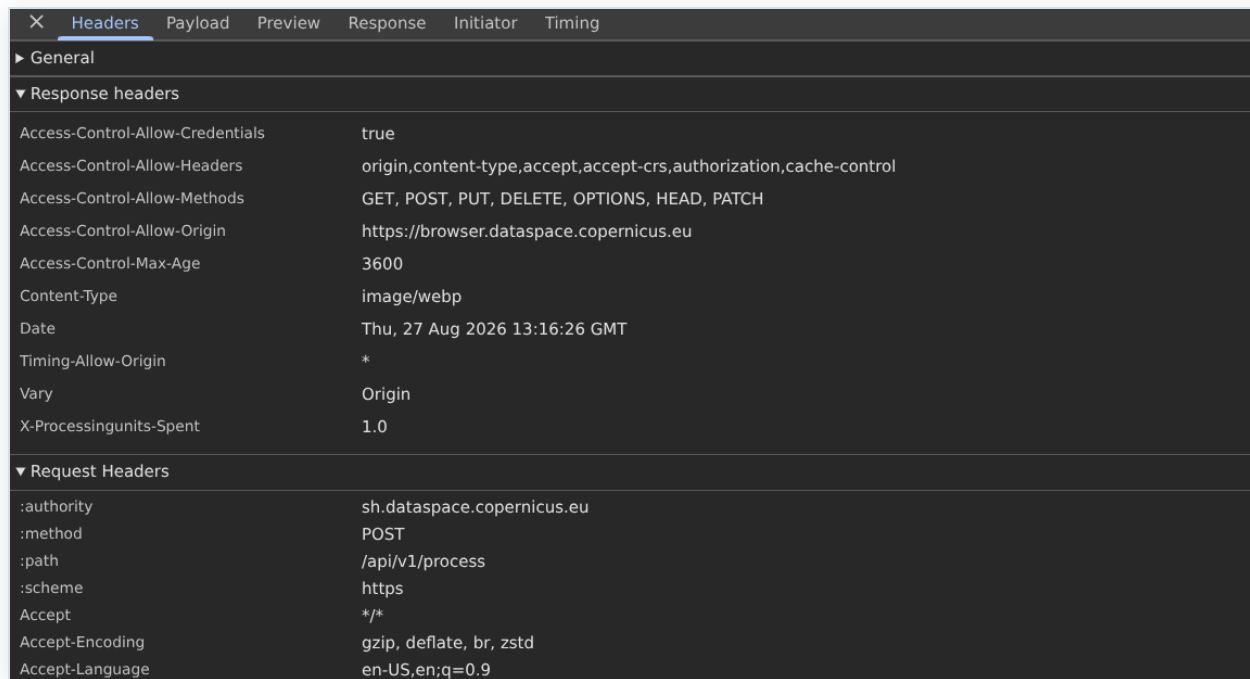
The script in the editor, and the same script in the request body

- Every tile on that map was a **request**
- The Browser is a **client**. There is nothing in it you cannot do yourself
- **Flip to a custom script and the endpoint changes.**

Preset tiles go to `openeosh.../1.2/result`. That is **openEO**, a **second processing API on CDSE**, with Sentinel Hub executing the graph. A custom script goes straight to `POST .../api/v1/process`. Two doors, and the app picks one for you.

And it tells you what it cost

Response headers,
on that same request.



X Headers Payload Preview Response Initiator Timing	
General	
Response headers	
Access-Control-Allow-Credentials	true
Access-Control-Allow-Headers	origin,content-type,accept,accept-crs,authorization,cache-control
Access-Control-Allow-Methods	GET, POST, PUT, DELETE, OPTIONS, HEAD, PATCH
Access-Control-Allow-Origin	https://browser.dataspace.copernicus.eu
Access-Control-Max-Age	3600
Content-Type	image/webp
Date	Thu, 27 Aug 2026 13:16:26 GMT
Timing-Allow-Origin	*
Vary	Origin
X-Processingunits-Spent	1.0
Request Headers	
:authority	sh.dataspace.copernicus.eu
:method	POST
:path	/api/v1/process
:scheme	https
Accept	*/*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9

- The request was **512 × 512**, three bands plus dataMask
- The response says **X-Processingunits-Spent: 1.0**
- That is the definition from the opening slide, coming back from the platform
- **It is a receipt, not a quote. Price a big job by running a small area first.**

X-Processingunits-Spent, returned on every call

⚠ Only the **Sentinel Hub** APIs meter in processing units.
openEO bills credits, and downloads are metered as transfer.

Two jobs

Six APIs. Every one of them does one or the other.

THE JOB

THE APIS

WHAT YOU GET

Find the data

STAC, OData, SH Catalog

Which scenes exist, and where

Process the data

Processing, Statistical, openEO

Pixels and numbers, computed server-side

The Sentinel Hub doors

The ones you use for the rest of today.

API	WHAT IT DOES	TODAY
Processing	<i>"provides images based on satellite data". This is where your evalscript runs</i>	Behind every tile today
Statistical	<i>"statistics calculated based on satellite imagery without having to download images"</i>	Used it in Exercise 1

Neither is new to you. **Statistical Info** gave you that greenest-date chart, and **Sentinel Hub has rendered every tile you have looked at**. For a preset the Browser asks through openEO; for a custom script, straight through Processing.

There is a third, **Catalog**: STAC over the ten collections you can process. Worth knowing it is there; we do not open it today.

Before the break, **break** something

① Click `</>` on the layer you have open

② Choose the **Custom script** radio.
It defaults to OpenEO, which is not what you want

③ In the **return** line, find the three band names.
Swap two of them

④ **Apply**

The colours change because you changed which band goes where.
That is the whole idea, and after the break you write one from nothing.

FIFTEEN MINUTES

Break

Everything before the break was **clicking**.

Everything after the break, **you write**.

Back in **15 minutes**

What an evalscript is

A function that runs once per pixel, on their machines.

- You do not download the scene.
You send **eight lines of JavaScript** to it
- The service runs your function **per pixel** and returns only the result
- Two functions: **setup()** declares what you want, **evaluatePixel()** decides what comes back
- **This is the whole idea, and it is the same one at every scale today.**

Today you write the first two. The last two are the same shape, with fitted coefficients or a threshold in the middle.

EVALUATEPIXEL CAN BE	A REAL ONE FROM THE CDSE REPO
A composite	True colour. Three bands, a picture
A spectral index	NDVI. One number per pixel
A regression	Water depth, $SDB = m1 \times ratio - m0$, in metres
A classification	Plastic greenhouses. Forest disturbance

You have been running one **since ten past two**

```
function setup() {  
  return {  
    input: ["B04", "B08", "SCL", "dataMask"],  
    output: [  
      { id: "default",      bands: 4 },  
      { id: "index",        bands: 1, sampleType: "FLOAT32" },  
      { id: "browserStats", bands: 2, sampleType: "FLOAT32" },  
      { id: "dataMask",     bands: 1 },  
    ],  
  };  
}
```

- default is the map tile
- index is the value beside your marker
- browserStats is the chart from Ex. 1
- dataMask is why nodata is transparent
- **You have already seen all four.**

The NDVI preset, opened with </>. Eighty lines. This is the first eleven.

Now delete all of it

Select the eighty lines.
Delete them. Type this.

```
//VERSION=3
function setup() {
  return {
    input: ["B04", "B03", "B02"],
    output: { bands: 3 }
  };
}

function evaluatePixel(s) {
  return [s.B04, s.B03, s.B02];
}
```

- **setup()** says which bands you want and how many come back
- **evaluatePixel()** runs once per pixel and returns what you want drawn
- That is the whole language. Everything else is arithmetic
- **Press Apply. It works, and it looks wrong.**

Why it came out dark

The bands are reflectance,
and reflectance is small.

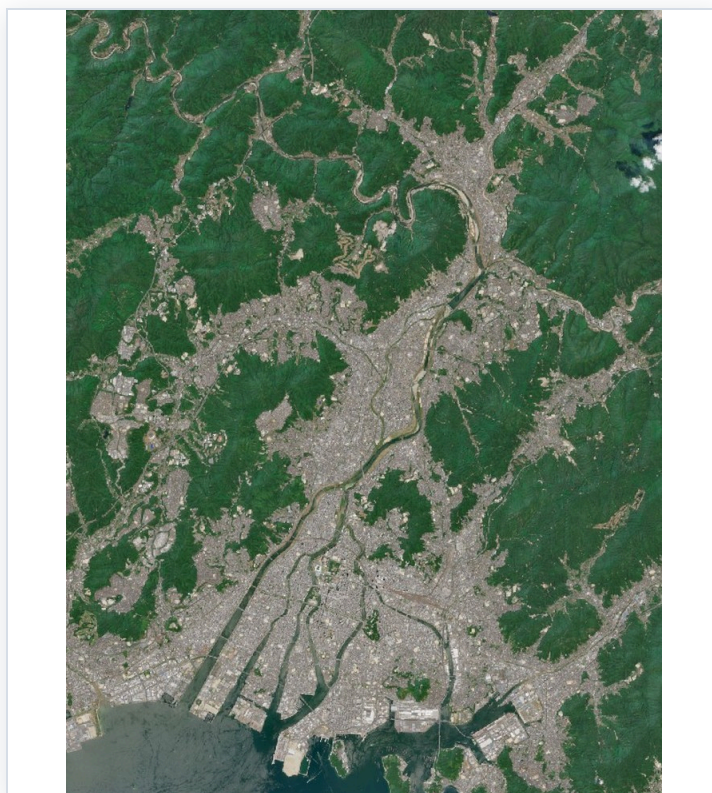
SAMPLETYPE	EXPECTS	YOU USE IT FOR
AUTO (default)	0 to 1 , stretched to 0–255	Pictures
UINT8	0 to 255	Pictures, when you do the stretching
FLOAT32	No limits	Indices , which go negative

- Sentinel-2 reflectance sits around **0 to 0.4**, not 0 to 1
- So [B04, B03, B02] tops out near $0.4 \times 255 \approx 102$,
which is why it is dim rather than broken
- **Multiply by 2.5 and 0.4 becomes 1.0. That is the whole trick.**

Ask for units: "DN" instead and the same bands arrive as **0 to 4000**,
at which point multiplying by 2.5 would be nonsense. **Know which unit you asked for.**

Same three bands, different picture

The count is the container.
The names are the picture.



[B04, B03, B02]
red, green, blue



[B08, B04, B03]
near-infrared where red used to be

- **1 band** comes back **grey**
- **3** is a picture
- **4** adds an **alpha channel**, so dataMask makes nodata transparent
- **Your NDVI returns one band.**
It will be grey, and that is correct.

From a picture to a measurement

Two edits, and it stops
being a photograph.

```
function setup() {  
  return {  
    input: ["B04", "B08"],  
    output: { bands: 1 }  
  };  
}  
  
function evaluatePixel(s) {  
  return [(s.B08 - s.B04) / (s.B08 + s.B04)];  
}
```

- **setup()** changed: swapped the visible bands for **red and near-infrared**, one output band instead of three
- **evaluatePixel()** changed: arithmetic instead of a list
- No `sampleType`, so AUTO stretches **0 to 1** onto the greyscale. Negative NDVI clamps to black, which over water is fine
- **Same two functions. Different question.**

● Do **not** add `sampleType: "FLOAT32"` here. The Browser asks for `image/webp`, and *"format image/webp does not support sample type FLOAT32."*
Float is for the API, where you ask for a TIFF back.

You do not have to write it all

The runtime ships helpers.

```
const ramp = new ColorRampVisualizer([
  [-0.2, 0x0000ff],
  [ 0.0, 0xffffffff],
  [ 0.8, 0x006400],
]);

function evaluatePixel(s) {
  return ramp.process(index(s.B08, s.B04));
}
```

Grey NDVI becomes a coloured map. `index()` is the normalised difference.

WHAT IS THERE

Visualizers	ColorRampVisualizer, ColorMapVisualizer
Maths	index, valueMap, combine, inverse
Colour	int2rgb, rgb2int, normalizeRGB
Ramps	blueRed, redTemperature, greenWhite
Quality flags	decodeS30LCIQualityFlags, decodeL8C2Qa

 **ramp.process()** returns three numbers, so a ramp needs output: `{ bands: 3 }`.

Carry bands: 1 over from the last slide and the map stays grey.

Worked examples: documentation.dataspace.copernicus.eu/APIs/SentinelHub/Evalscript/Utilities.html

So what about the clouds?

The classification from the first hour, now as three lines of code.

```
input: ["B04", "B08", "SCL"],
output: { bands: 4 }

...
function evaluatePixel(s) {
  if ([3,      // cloud shadow
      8,      // cloud, medium probability
      9,      // cloud, high probability
      10     // thin cirrus
    ].includes(s.SCL)) {
    return [1, 0, 0, 1];
  }
  return [...ramp.process(index(s.B08, s.B04)), 1];
}
```

- You can ask for bands you never draw. SCL carries a class number, not a colour
- Branch on it. **Paint the rejects red** so you can see exactly what you threw away
- Same trick works on CLD, dataMask, QLEVEL on CLMS, localIncidenceAngle on Sentinel-1
- **You are not filtering the scene. You are deciding, per pixel, what counts.**

SCL is the **Scene Classification Layer**, computed by ESA and shipped with the scene. Red is [1, 0, 0], not [255, 0, 0]: AUTO works in **0 to 1**.

⚠ Add 7 and you also drop *unclassified*, which over bare soil is a lot of ground.

The same editor, on CLMS

Evalscripts are not a Sentinel-only thing.

```
input: ["LSWT", "dataMask"],
output: [ ...the same four as the preset... ]
const ramp = new ColorRampVisualizer([
  [ 0, 0x0000ff],    // cold
  [20, 0xffffffff],  // neutral
  [30, 0xff0000],    // warm
]);
const celsius = s.LSWT * 0.01;    // Kelvin would be + 273.15
return {
  default:    [...ramp.process(celsius), s.dataMask],
  index:      [s.dataMask ? celsius : NaN],
  browserStats: [celsius, s.dataMask],
  dataMask:   [s.dataMask],
};
```

- Every CLMS product is a BYOC collection, and BYOC collections take custom scripts
- The official script returns **Kelvin**.
The offset cancels, so Celsius is one multiplication
- index puts the number **beside your marker**.
browserStats feeds **Statistical Info**
- **The same four outputs as the NDVI preset.**
Different collection, identical shape.

lswt_nrt_global_1km_10daily_v1. ⚠️ FLOAT32 works on index and browserStats, but never on the rendered default.

Two collections, one script

Data fusion. Worth knowing exists, not worth learning now.

```
input: [  
  { datasource: "s1grd", bands: ["VV", "VH"] },  
  { datasource: "s2l2a", bands: ["B04", "B08", "SCL"] },  
]  
...  
function evaluatePixel(samples) {  
  const s1 = samples.s1grd[0];  
  const s2 = samples.s2l2a[0];  
  
  return cloudy(s2.SCL) ? fromRadar(s1)  
                        : fromOptical(s2);  
}
```

- input becomes a **list of collections**, each tagged
- Works on Process, Statistical and Batch.
No special account. It just costs more units
-  And the trap: `samples` becomes an **array**, so `samples.s2l2a[0].SCL`, not `samples.SCL`
- **That is the opposite of everything on the last four slides.**

Six worked examples in the docs: NDVI from **S1 + S2**, **ship detection**, built-up area from three sources, fire monitoring from the same collection at two dates.

 The array trap is also what an AI gets wrong when you ask it to fuse. **Remember it when you start prompting.**

Turning it on

In the Custom script panel, under the radio.

Composite Index Custom

☐ OpenEO process graph *i* ☒ Custom script *i*

☒ Use additional datasets (advanced)

Primary dataset: S-2 L2A

Datasource alias:

Mosaicking order:

Additional datasets:

^	s1grd (S-1 GRD)	🗑
Datasource alias:	s1grd	
Mosaicking order:	Most recent	
Orthorectification:	Yes (Copernicus 10/30...)	
Polarization:	DV	
Acquisition mode:	IW	
Resolution:	HIGH	
Speckle Filter:	NONE	
Backscatter coefficient:	GAMMA0_ELLIPSOID	

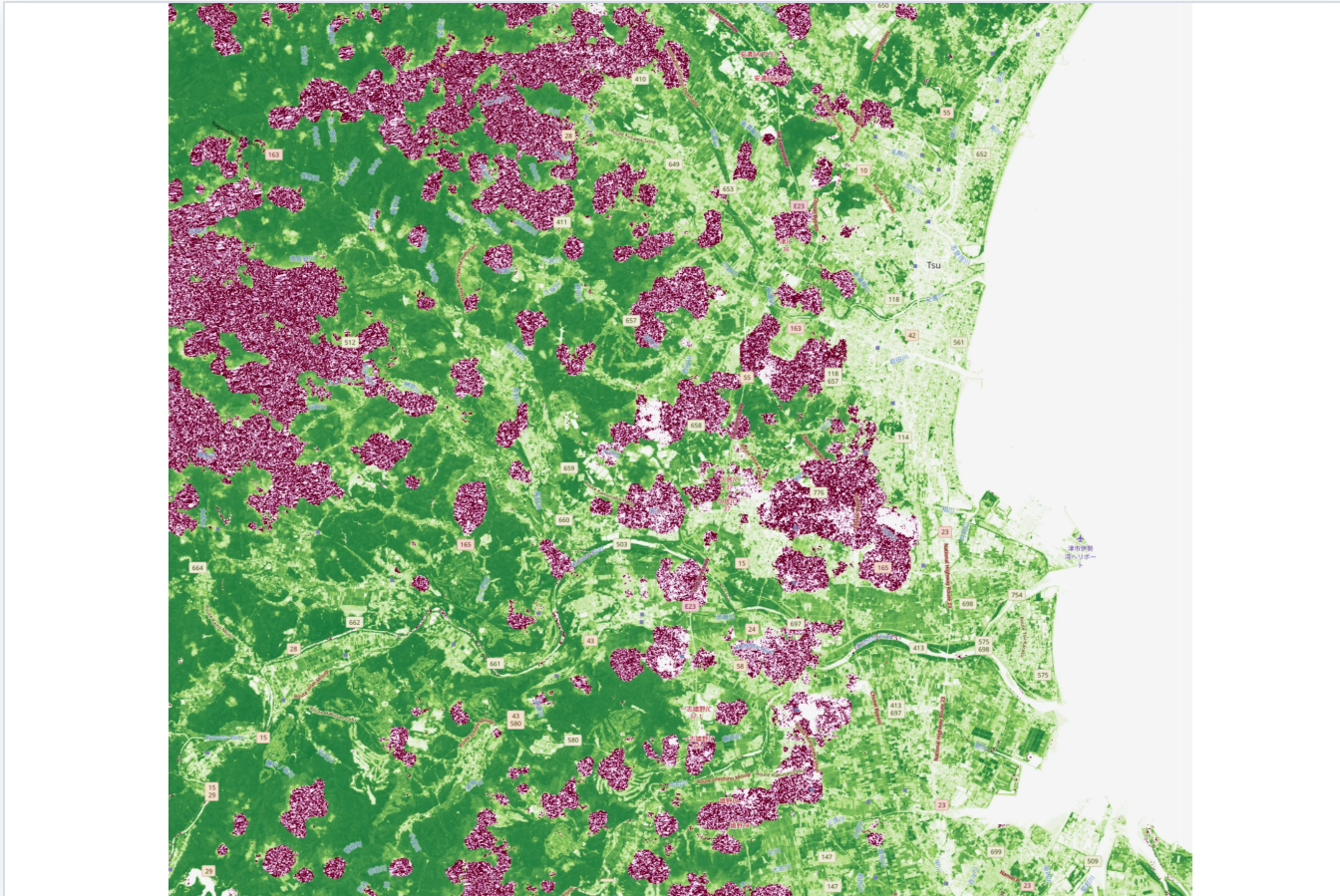
☐ Customize timespan

S-1 GRD

- Tick **Use additional datasets (advanced)**
- Each dataset gets a **Datasource alias**.
That string is what your script has to say
- Orthorectification, polarisation, **speckle filter**:
all set here, none of it in the evalscript
- **The alias is the contract. Type `s1grd` in the box, write `samples . s1grd` in the code.**

Where the sky was closed

Two collections, one script, one branch.



Tsu, Mie. Cloud on this date, filled from radar.

- **Green:** NDVI, measured by Sentinel-2
- **Magenta:** Sentinel-1 backscatter, standing in wherever SCL said cloud
- Same colour brightness, different hue, so you can see which sensor answered
- **Nothing here is clever. It is one *if*, over two collections.**

Your turn

Open `</>`, Custom script radio, and experiment.

- Swap the bands. [B08, B04, B03] is false colour
- Make it an index, then colour it with a ramp
- Move the ramp stops and watch what appears
- **No target. Get comfortable with the pieces from the last six slides.**
- Gate on **SCL** and paint what you reject
- Point it at a **CLMS** collection instead
- Add a second collection with **Use additional datasets**

Change one thing, Apply, look. The error messages are the useful part.

STOP 4 OF 5

AI

You know what is in the catalogue. You can read an evalscript.

That is what makes the next forty minutes work.

Why this works **now**

- You have seen **what the catalogue holds**, and where it stops
- You have written an evalscript by hand, so you know **roughly what one should look like**
- Evalscripts suit an **LLM** unusually well: **documented**, plain **JavaScript**, small inputs and outputs, and **you can see** whether it worked
- **It gets you to a working script in a minute. Judging it is still yours.**

Open a chatbot cold, with neither of those two things, and you cannot check the answer. That is the difference between this and a magic trick.

Point it at the **right** page

It can read documentation. It cannot know which documentation is yours.

- There are **two script collections** and two band vocabularies.
A model will happily mix them
- Ask cold for an evapotranspiration script and you may get CLP.
That band does not exist on CDSE. The code looks right and returns nothing
- Give it **the product page** for the collection you are actually on
- **You are not correcting the model. You are telling it which collection it is writing for.**

Naming the collection is usually enough to stop the guessing.

The list you got at the break has the bands for every CLMS product, with type, range and scaling.

Watch this

Plain language in. Working script out. Not on the first try.

① Ask for what you want, in a sentence

② Paste the result into `</>`, press **Apply**

③ **It breaks**

④ Tell it what happened. Paste the band list, or point it at a script that already works

⑤ It works

The loop is the point. You can [see](#) whether it worked, which you cannot do with most code an AI writes for you.

 A failed request gives a [missing tile and an error panel](#). A [black image](#) is a request that succeeded and returned zeros. Those are different problems.

Somebody has **already** written it

eu-cdse/sentinel-hub-custom-scripts

WHERE	SCRIPTS
clms/	448, one folder per product
landsat/	16
sentinel-2/	15
sentinel-3/, dem/, mosaics	6

- 485 evalscripts, GPL-3.0, organised by mission
github.com/eu-cdse/sentinel-hub-custom-scripts
- Every product page in the documentation links to its own folder, which is how you find the one you need
- `contribute/example/` is the template. You can open a pull request
- **Read three before you write one. It is faster than any prompt.**

It is also the best context you can give an LLM. Name the repo and the folder for your collection, and an agent that can reach GitHub will pull in the scripts it needs.

Stuck for an idea?

Three ways in.

- **Restyle** something you saw today: same data, a different look
- **Port** a script to a collection nobody wrote one for, or **minify** one for a sharing URL
- **Implement a paper.** Five that work on CDSE data:

THE INDEX	WHAT YOU GET	BANDS
kNDVI	Vegetation, where plain NDVI flattens out	B08, B04
PI	Floating plastic and debris on water	B08, B04
ABAI	Burned area, with a threshold the paper publishes	B12, B11, B03
SDWI	Open water through cloud, at night	VV, VH
Snow and ozone	Grain size, and the ozone column overhead	B01, B03, B8A

- **50+ come with the exercise. Formulas checked against the papers, not against your area.**

Or browse: [awesome-spectral-indices](#) lists 280, with formulas, bands and DOIs.
github.com/awesome-spectral-indices. Transcriptions, so follow the DOI when it matters.

Now you

Your own question. Your own place.

- **Ask the thing you came here wanting to see.**
The gap you hit before the break is the best brief you have
- **It will break.** That is the exercise, not a setback
- Stuck for a topic? The slide before this one is a menu:
restyle, port, or a paper from the handout
- **Or answer a question the catalogue alone cannot.**

Paste the band list from the docs **before** it guesses.

When it breaks, tell it **what you saw**, not *"it does not work"*.

 Ask for **two collections** and it will get samples wrong.
You saw why when we took an evalscript apart.

Show us

Three ways to get it off your laptop.

HOW

WHEN TO USE IT

Share button in the Browser

Short scripts. The script travels **inside the URL**

A GitHub gist, raw URL

Long scripts. The Browser loads it via `evalscriptUrl`

A pull request

When it is good enough that someone else should have it

- Paste a link, and we will open two or three on the projector
- **What did you ask for, and what did it get wrong first?**

⚠ Read it before you submit it. CDSE's own community line:
"detailed human review is essential, nobody likes fully AI-generated PRs."

STOP 5 OF 5

Python

The Browser has been sending your script to an API all afternoon.
Now you send it yourself.



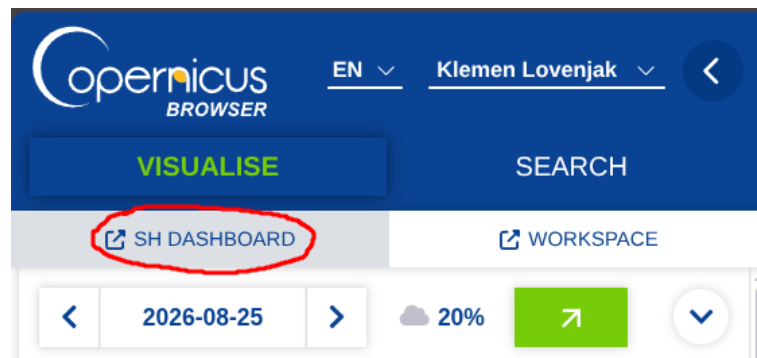
Same script, **different envelope**

This part is a demo. The last four minutes are not.

- The evalscript is **portable**. Browser or API, it is the same text every time
- This is an ordinary Jupyter notebook, and it is in the leave-behind
- **`pip install sentinelhub`**. One dependency, and it is yours.

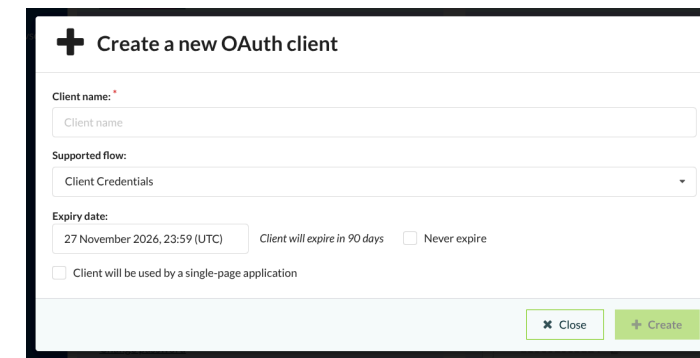
Where the **account** shows up

Notebook §1. An **OAuth** client from the dashboard, and a token.



① Top of the panel, in the Browser you already have open

- This is the only place the free account becomes visible in code
- ⚠ **The secret is shown once**, and the default expiry is **90 days**
- **Processing is accounted for. That is the whole point of this cell.**



② **User settings**, foot of the sidebar
→ **OAuth clients** → **Create**

The same 10 000 requests and 10 000 processing units a month that you have been spending all afternoon without once being asked about them.

Your NDVI, from Python

Process **API**. The script
from Exercise 2, unchanged.

- A bounding box, a resolution, a collection, and **the evalscript field**
- What comes back is **the whole grid**, one value per pixel.
For this box that is about **a million numbers**
- **You already wrote this. It is the same text in a different envelope.**

You watched this go past in devtools **before** the break,
and you edited one by hand **after** it. **Same script, three envelopes.**

2.28 gigabytes, or four kilobytes

Both are free and supported.
The question is when, not which.

WHAT YOU ASK FOR

WHAT ARRIVES

One S2 L2A product over Hiroshima

2.28 GB across 43 assets

Just B04 and B08

240 MB

A year of NDVI over one field

166 GB, or 17.5 GB band-picking

The same year, as numbers

One request. Kilobytes

- Downloading is often **the right answer**. S3 keys are self-service and the free tier is **12 TB a month**
- **Whole products, every band, your own atmospheric correction: download, and nobody should talk you out of it.**

You already did the cheap version in Exercise 1, in four seconds, without knowing what it would have cost the other way.

One area, one year

Statistical API. Notebook §5.

- Same evalscript, **no pixels come back at all.**
Per interval: **min, max, mean, stDev**, and the counts
- **The last slide came back with a million numbers. This one comes back with twelve.**

WHO DECIDES WHAT COUNTS AS CLOUD

SCL CLASSES THROWN AWAY

Browser **NDVI** preset

8, 9, 10. Ignores shadow

This notebook

3, 8, 9, 10, 11. Shadow first

Official quarterly mosaic

1, 3, 7, 8, 9, 10. The strictest

CLMS products

none. Decided upstream; you get QLEVEL

- **All four are CDSE. None is wrong. No-data is a fact. Cloud is an opinion.**

Watch which spikes disappear when the shadow mask goes on. Those **survived the Browser's cloud slider** in the first half hour.

Ten lakes, one year

Statistical API, ten times over.

What a browser tab cannot do.

```
// lake surface water temperature, in Celsius
var valid = (s.dataMask === 1 && s.QLEVEL >= 4) ? 1 : 0;
return {
  celsius: [ s.LSWT * 0.01 ],
  dataMask: [ valid ],
};
```

The official script adds 273.15. Drop it and you have Celsius.

The **Batch** Statistical API takes a **GeoPackage** instead, up to **700 000 polygons** in one job, written straight to object storage. That is how region, country and continental scale gets done.

⚠ Copernicus Service account only, and **you do not need it for ten**.

- Statistical API takes **one geometry per request**, so ten lakes is **ten requests, in a loop**
- P10D intervals, because the product is 10-daily. A year is about thirty-six points per lake
- **Ten requests out of ten thousand a month. The loop is the whole trick.**

github.com/lovenjak/hands-on-cdse

The slides, the notebook,
the collection list.

- **Everything you did today was on the free tier.**
Every request, every script, every chart in the notebook
- No contract, no quota upgrade, no institutional account
for anything you saw
- **It is all still there tomorrow, and it is all still free tomorrow.**

Where to go next

Four places, and none of them needs anything from us.

WHAT

485 official evalscripts, one folder per product

Where the team and other users answer

GUI to runnable Python

Bulk download, 12 TB a month

- `ask.copernicus.eu` is a **bot**. Useful for looking things up, but the forum is where a real answer comes from
- **Write a script worth having and open a pull request. That collection is how it grows.**

WHERE

`github.com/eu-cdse/sentinel-hub-custom-scripts`

`forum.dataspace.copernicus.eu`

Requests Builder

S3 keys, self-service

Imagery throughout: **contains modified Copernicus Sentinel data and Copernicus Service information.**

Europe's **Sovereign Infrastructure** for open EO

2026-09-03 11:30, Room 5.

- Petabytes of imagery, derived products and land monitoring, all through **cloud-native APIs**
- What **sovereign EU infrastructure and open standards** make possible on top of that
- **Today was the *how*. That one is the *why*.**

You gave up a Sunday afternoon for this. Thank you.